

AMPL

AMPL è un linguaggio progettato per descrivere e risolvere complicati problemi di programmazione matematica, come ad esempio problemi di ottimizzazione.

Non risolve in modo diretto questi problemi, ma chiama dei solver (come CBC, GURUBI, CPLEX) per ottenere le soluzioni.

Avendo un problema di programmazione lineare andremo a separare due concetti:

- **Modello** → avrà estensione **.mod** → descrive la struttura logica del modello (indici, variabili di decisione, funzione obiettivo e vincoli)
- **Dati** → avrà estensione **.dat** → contiene i valori numerici del problema

N.B: Alcune cose da sapere prima di procedere:

- è possibile aggiungere dei **commenti**, introducendo il testo dal simbolo #
- le istruzioni sono separate dal simbolo ;
- AMPL **distingue le maiuscole dalle minuscole**, cioè se io scrivo
$$\begin{array}{l} \text{var } x1; \\ \text{var } X1; \end{array}$$
sto introducendo due variabili distinte
- Ogni riga ha **lunghezza massima di 255 caratteri**
- Si può esprimere un **insieme di valori ordinati attraverso i suoi estremi** separati da ... (es: *set indice = 1 ... 5;*)

ESEMPIO:

Avendo il seguente problema di programmazione lineare:

$$\begin{array}{ll} \min & 4x_1 + 2x_2 - 3x_3 \\ & 2x_1 + 3x_2 + x_3 \leq 6 \\ & -4x_1 + 3x_2 - x_3 \leq 12 \\ & x_1 \leq 0, x_2 \geq 0, x_3 \geq 0 \end{array}$$

Un programma AMPL, come detto in precedenza, si divide in:

- **Modello:**

Nel modello troviamo gli insiemi, i parametri, le variabili, funzione obiettivo

- Gli insiemi li introduciamo con la parola chiave **set** → Questi definiscono il dominio del problema e la sua dimensione. Nel nostro esempio avremo:

$$\begin{array}{ll} \text{set J1;} & \# \text{variabili} \leq 0 \text{ (cioè } x_1 \text{)} \\ \text{set J2;} & \# \text{variabili} \geq 0 \text{ (cioè } x_2 \text{ e } x_3 \text{)} \end{array}$$

set J; *#variabili (cioè x_1, x_2 e x_3)*
set I; *#insieme vincoli*

- I parametri verranno successivamente quantificati nel file dati, in modello scriviamo in modo “generico”

param alfa;
param b {I}; *#coefficienti dei termini noti (cioè 6 e 12)*
param c {J}; *#coefficienti della funzione obiettivo (cioè 4, 2, -3)*
param a {I, J}; *#coefficienti delle variabili nei vincoli*

- Le variabili sono le grandezze che descrivono la soluzione del problema. Il loro valore deve essere determinato dal risolutore.

var x { J }; *#variabile funzione obiettivo (solo la x)*

- La funzione obiettivo specifica la grandezza del problema di cui si vuole trovare il valore ottimale. Viene introdotta dalla parola chiave ***minimize*** o ***maximize***, seguita da un ***nome*** (obbligatorio), dal simbolo ***:*** e ***dall'espressione che definisce la funzione*** obiettivo in termini dei dati e delle variabili. E dopo vengono inseriti i vincoli con la parola chiave ***subject to*** o anche ***s.t.***

#funzione obiettivo

minimize z: sum { j in J } c [j] * x [j];

#vincoli (ne inseriamo solo uno perchè entrambi hanno lo stesso segno)

subject to v1 { i in J }: sum { j in J } a [i, j] * x [j] <= b [i] + alfa

#vincoli di segno (v2 rappresenta il $\leq$ e v3 rappresenta il $\geq$)

subject to v2 { i in J1 }: x [j] <= 0;

subject to v3 { j in J2 }: x [j] >= 0;

N.B: alcuni comandi di operatori

Operatore	Simbolo
Potenza	$\wedge$ (oppure **)
Numero negativo	-
Somma	+
Sottrazione	-
Prodotto	*
Divisione	/
Divisione intera	div
Modulo	mod
Differenza non negativa (max(a-b,0))	less
Sommatoria	sum
Produttoria	prod
Minimo	min
Massimo	max
Unione di insiemi	union
Intersezione di insiemi	inter
Differenza di insiemi	diff
Differenza simmetrica di insiemi	symdiff
Prodotto cartesiano di insiemi	cross
Appartenenza a un insieme	in
Non appartenenza a un insieme	notin
Maggiore, Maggiore o uguale	$i \geq j$
Minore, Minore o uguale	$i \leq j$
Uguale	$=$ (oppure ==)
Diverso	$\neq$ (oppure !=)
Negazione logica	not (oppure !)
And logico	and
Quantificatore esistenziale	exists
Quantificatore universale	forall
Or logico	or (oppure —)
If then else	if then else

Operatore	Simbolo
Valore assoluto	abs(x)
Arco seno	asin(x)
Arco seno iperbolico	asinh(x)
Arco coseno	acos(x)
Arco coseno iperbolico	acosh(x)
Arco tangente iperbolica	atanh(x)
Seno	sin(x)
Seno iperbolico	sinh(x)
Coseno	cos(x)
Coseno iperbolico	cosh(x)
Tangente	tan(x)
Tangente iperbolica	tanh(x)
Approssimazione intera per difetto	floor(x)
Approssimazione intera per eccesso	ceil(x)
Logaritmo naturale	log(x)
Logaritmo decimale	log10(x)
Esponenziale	exp(x)
Radice quadrata	sqrt(x)
Minimo fra più numeri	min(x1,x2,...,xn)
Massimo fra più numeri	max(x1,x2,...,xn)

Operatore	Significato
not	(not a) è vera quando a è falsa, e viceversa
and	(a and b) è vera quando sono vere sia a sia b
or	(a or b) è vera quando almeno una fra a e b è vera

- **Dati:**

I dati sono i valori numerici che definiscono in dettaglio il problema che si vuole affrontare, una volta precisata la sua struttura logica.

Nel file dati inseriamo prima gli insiemi e poi andiamo ad elencare i parametri:

- Gli insiemi nel nostro caso:

```
set J1 := x1;           #variabili  $\leq 0$  (cioè  $x_1$ )
set J2 := x2, x3;       #variabili  $\geq 0$  (cioè  $x_2$  e  $x_3$ )
set J := x1, x2, x3;    #variabili (cioè  $x_1, x_2$  e  $x_3$ )
set I := r1, r2;        #insieme vincoli

param alfa := 5;        #a scelta

param b :=              #termini noti
r1 6;
r2 12;

param c :=              #coefficienti funzione obiettivo
x1 4;
x2 2;
x3 -3;

param a :=              #coefficienti vincoli
      x1    x2    x3 =
r1    2     3     1
r2   -4     3    -1;
```

- **File Run:**

Infine scrivi un terzo file *.run* come ad esempio:

```
model esercizio_1.mod;           #richiama il file .mod

data esercizio_1.dat;            #richiama il file .dat

option solver cplex; #scegliere il solver      #scegli il solver

for {i in 1...10} {

solve;                            #per risolvere il problema
```

```
let alfa := 1;  
display z;  
display x;  
display v1;  
display v1.slack;  
};
```

Per eseguire digita sulla console ***include nome_file.run;***